COSC204 Computer Systems

Lecture 24: Protection of file systems

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture: I/O systems

- Hardware: ports, buses, controllers
- Kernel I/O interface
- I/O performance

This lecture: protection of file systems

- Protection model
- Access control bits
- Access control models

Security implementation will use a protection model

- In any multiuser OS, such as Linux, file systems will be shared by many users.
- Typically, users can choose to share their files with others or keep them private.
- The protection model of a file system needs to define who can access what and enforce the defined policies.

There are many different types of filesystem access

File systems provide many different types of access permission:

- Read: read from a file
- Write: write to or modify a file
- Execute: load a file into memory and execute it
- Delete: delete a file from a directory and free its space
- List: show the name and attributes of a file

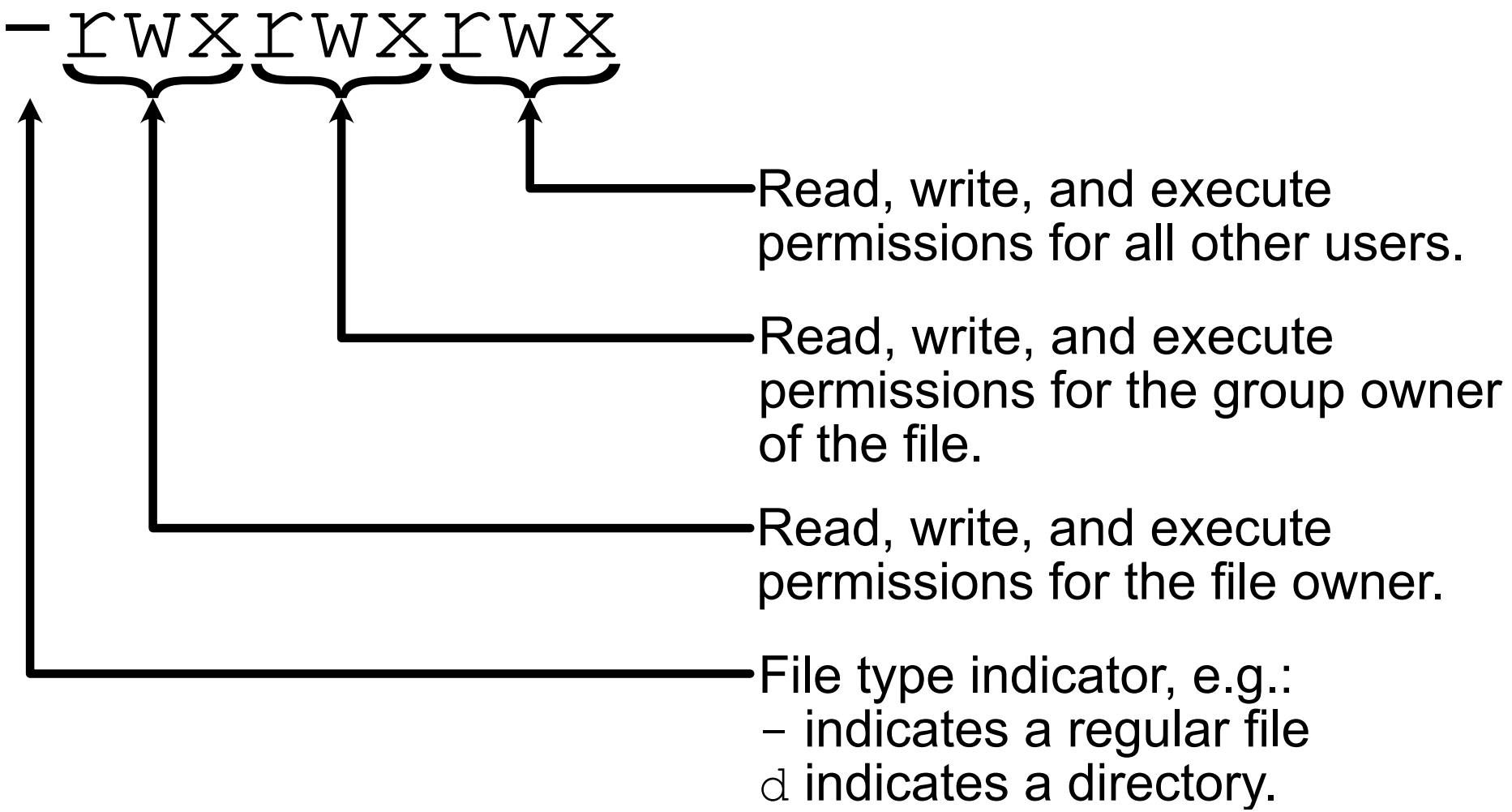
Typical Linux/Unix file system access control

The common approach to access control is based on the identity of the user. In Linux/Unix systems, the users are identified and grouped into sets in three different ways for each file:

- Owner: the user who created this file;
- Group: a single, named set of users; the names and members of these groups are controlled by the system administrators;
- Others: all users in the system.

When a file is created, it is automatically assigned the owner's user ID and a group ID.

The permission bits in Unix/Linux file systems



Permission bits in typical Unix/Linux filesystems

For each type of user selection: owner, group, others, there are three bits `rwx` to represent the permissions.

- `r` indicates whether the file is readable
- `w` indicates whether the file is writeable
- `x` indicates whether the file is executable

This gives 9 permission bits per file. Consider these permissions:

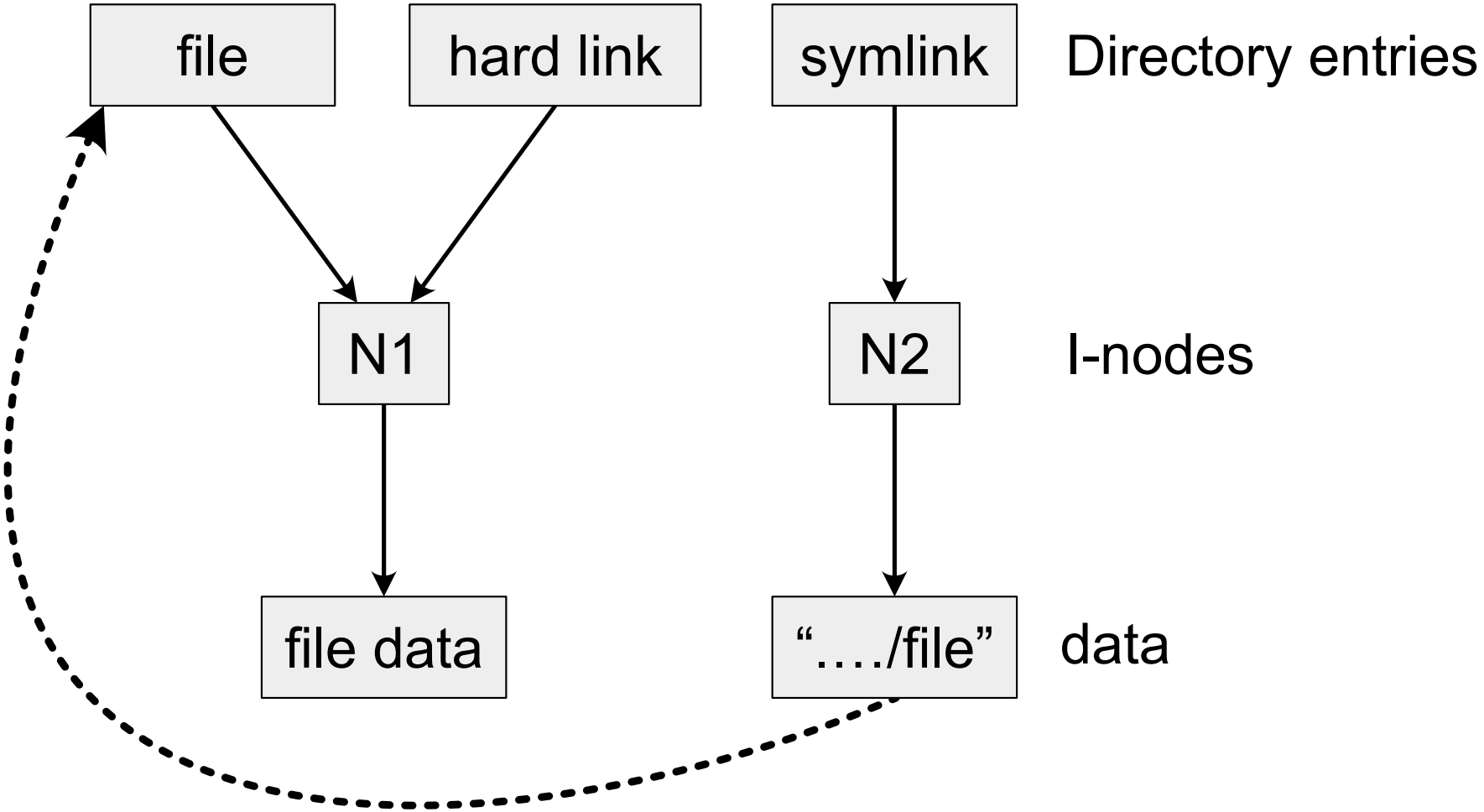
```
-rw-r--r-- 1 dme csstaff 217 Jun 22 14:52 Makefile
```

The first three bits are for the owner, the second three bits are for group access, and the last three bits are for other users.
Here, the owner can read and write, any member of group `csstaff` can read, and other users can read. The '–' means the corresponding permission is disabled.

There are further (more advanced) file attributes

- SetUID bit: An executable with this bit set will run with the privileges of the file's owner, rather than the user who is executing the file. Shown as `s` in the owner's execute position.
  - For example, if the owner of the file is `root` (superuser), when a normal user runs the file, the running process has the privilege of the superuser (which has very few access restrictions).
- SetGID bit: Similar SetUID bit except file is run with the privilege of the owner's group. Shown as `s` in the group's execute position.
- Sticky bit: A directory with this bit set restricts the deletion of files within it. Shown as `t` in the others' execute position.
  - A directory with permissions `rwxrwxrwx` lets anyone delete any files in that directory. The sticky bit `rwxrwxrwt` only allows users to delete their own files. The directory's owner can delete any file in it.

Soft links and hard links to files



There are many access control models, e.g., DAC

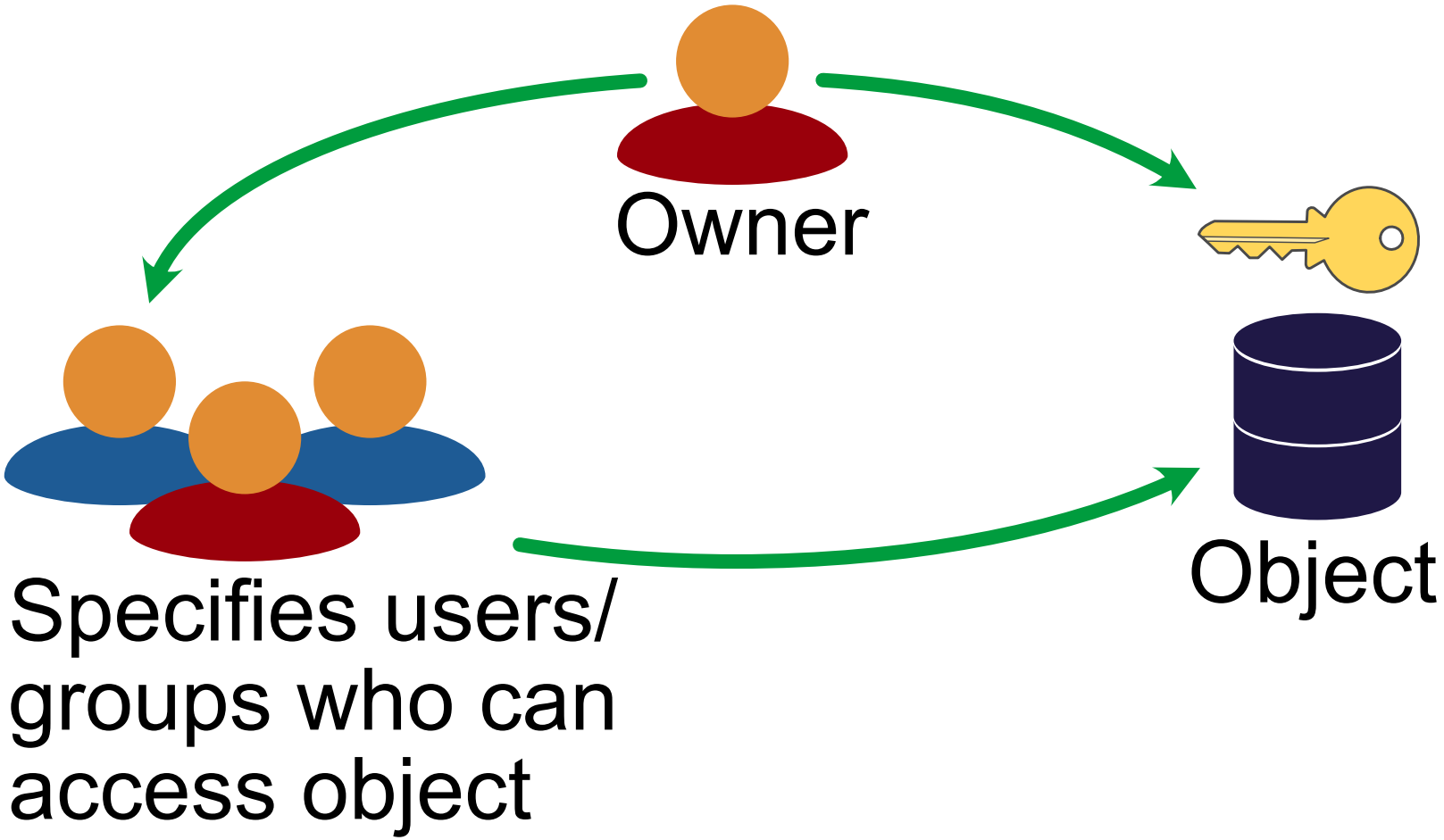
We will cover three common access control models:

- Discretionary Access Control (DAC)
- Mandatory Access Control (MAC)
- Role-based Access Control (RBAC)

First, Discretionary Access Control (DAC):

- The permissions of the object—e.g., a file—are under the control of the subject—e.g., the owner of a file.
- The owner typically selects the other subjects that can access the object and the type of access that they are allowed.
- Thus non-owners' access is at the discretion of the owner.

Depicting Discretionary Access Control (DAC)

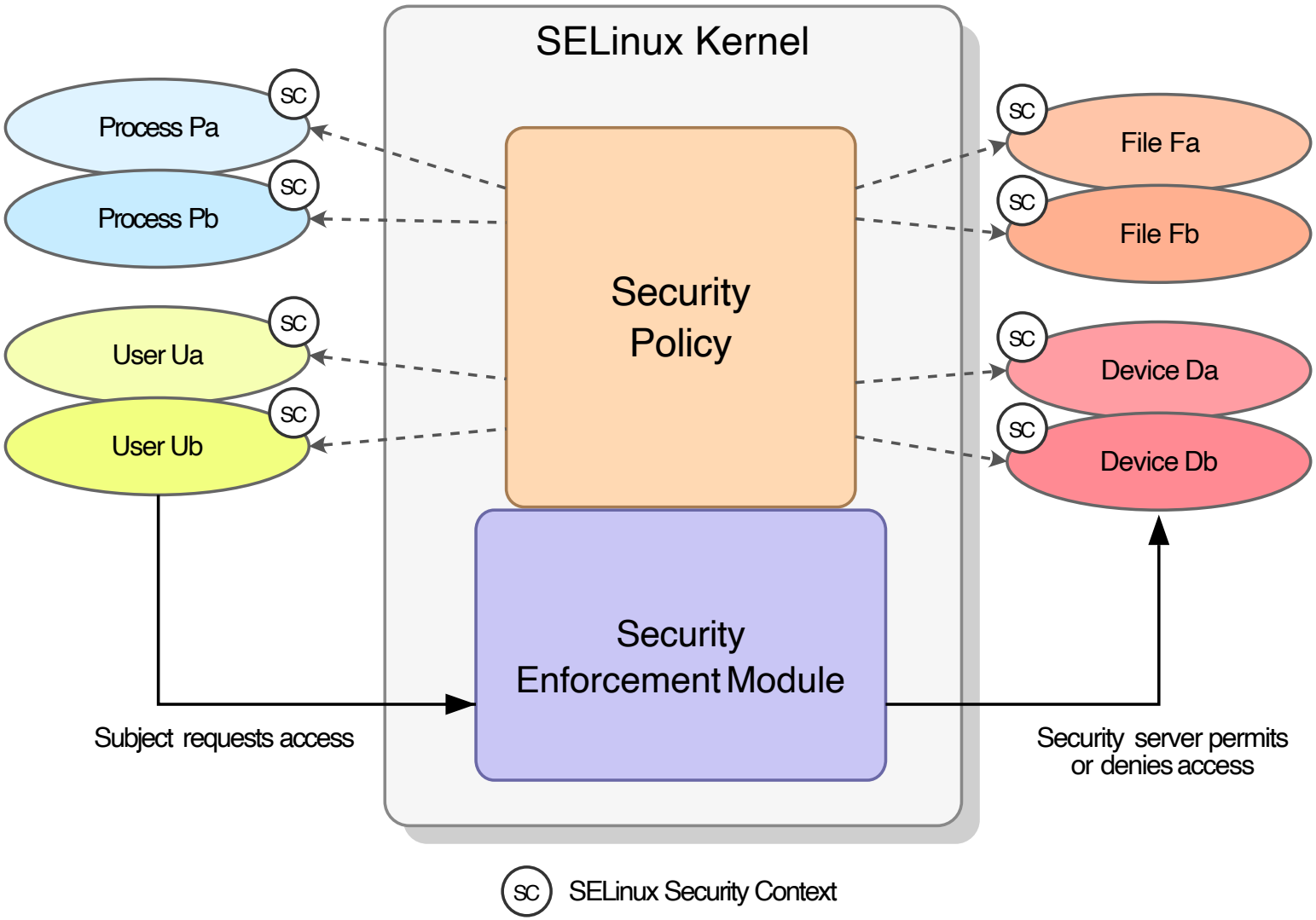


A more strict access control model: MAC

Mandatory Access Control (MAC):

- Permission of a subject to access an object is determined by a central authority. (The application of the policy is mandatory, and thus not at the discretion of the owner of the object.)
- Usually objects are assigned a sensitivity label (e.g., 'Top Secret', 'Secret', 'Confidential', 'Unclassified') and subjects are assigned a clearance level on the same scale.
- Typically MAC systems enforce two main rules:
  - No read up: a subject can only read an object if the subject's clearance is greater than or equal to the object's sensitivity.
  - No write down: a subject can only write to an object if the subject's clearance is less than or equal to the object's sensitivity.

Mandatory Access Control (MAC) in Linux



Access control models: RBAC

Role-based Access Control (RBAC)

- In RBAC, permissions are associated with roles, and users are assigned to appropriate roles.
  - A role is a collection of permissions that define the authority and responsibility conferred on a user assigned to the role.
  - A user can be assigned to multiple roles.
  - Multiple users can be assigned to a role.
- A user can exercise permissions associated with their active role.
  - Typically a user can change roles during a session.

Access control lists (ACLs) add flexibility

- An access control list (ACL) is a list of permissions attached to an object (file).
- The list specifies who or what is allowed to access the object and in what way.
- ACLs typically provide a more flexible permission mechanism than the traditional Unix permissions scheme.
  - In Linux filesystems, ACLs will often be attached in addition to the traditional permission bits.
- An ACL can be thought of as a list of tuples of the form (*subject*, *operation*) where each operation is one of read, write, execute, delete, etc.

Reading

For this lecture, relevant material comes from Chapter 10 (Section 6) and Chapter 14 (Section 6).